

🧠 AI Driven SQL Agent — Full Team PRD

University Project | Backend Only | Team of 4

How to use this document: Each team member reads **ONLY** their own section. Give this full document to an AI (like Claude, ChatGPT, or Gemini) and say: *"I am [YOUR NAME]. My module is [MODULE NAME]. Based on this PRD, generate my complete code."* The AI will write everything for you — because this document explains everything in full detail.

🕒 **DEADLINE: Saturday, 2nd May 2025 — 12:00 PM (Noon)** All branches must be pushed and ready for review by this time. No extensions.

📌 Project Overview

Project Title: AI Driven SQL Agent **Type:** Backend only (Flask REST API) **Goal:** Build a system where users can upload data files, ask questions in plain English, and get answers using AI.

GitHub Repo: <https://github.com/aman-ali65/AI-Driven-SQL-Agent>

What the system does (in plain English):

1. A user uploads a CSV or Excel file → the system saves it as a database
2. The user types a question like "How many students passed?" → AI converts it to SQL → runs it → returns the answer
3. A user uploads a PDF/PPT → asks questions from it → AI answers using the document
4. A user can create/modify/delete database tables using plain English
5. The system also generates insights from query results
6. When SQL query results contain data, the system **automatically generates charts** for easy visualization

📁 Full Project Folder Structure

```
ai-sql-agent/  
|  
├─ app.py                ← Main Flask entry point  
├─ requirements.txt       ← All Python dependencies  
├─ .env                  ← API keys (never commit this!)  
├─ .gitignore            ← Tells git to ignore .env and uploads  
├─ database/  
|   └─ data.db           ← SQLite database (auto-created)  
|  
└─ modules/
```

```
|   ├── sql_agent/                                ← AIZA's module
|   |   ├── __init__.py
|   |   ├── controller.py
|   |   ├── validator.py
|   |   └── agent.py
|
|   ├── file_to_db/                              ← HABIBA's module
|   |   ├── __init__.py
|   |   └── uploader.py
|
|   ├── rag_system/                              ← REHAN's module
|   |   ├── __init__.py
|   |   └── rag.py
|
|   ├── schema_insights/                        ← AQSA's module
|   |   ├── __init__.py
|   |   └── schema_agent.py
|
|   └── visualizer/                             ← SHARED module (Aqsa sets up, everyone uses)
|       ├── __init__.py
|       └── chart_generator.py
|
└── uploads/                                    ← Uploaded files stored here (CSV, PDF, etc.)
    └── embeddings/                            ← Stored numpy embeddings for RAG
```

🌸 GitHub Workflow — For Everyone

The repo already exists at: <https://github.com/aman-ali65/AI-Driven-SQL-Agent> You do NOT need to create a new repo. Just clone it and create your branch.

Step 1 — Clone the Existing Repo

Each member runs this once on their computer:

```
bash
git clone https://github.com/aman-ali65/AI-Driven-SQL-Agent.git
cd AI-Driven-SQL-Agent
```

Step 2 — Create Your Branch

Each person creates their own branch from `main`:

```
bash
```

```
# Create your own branch (each person runs their own line)
git checkout -b aiza/sql-agent      # ← Aiza runs this
git checkout -b habiba/file-to-db   # ← Habiba runs this
git checkout -b rehan/rag-system    # ← Rehan runs this
git checkout -b aqsa/schema-insights # ← Aqsa runs this

# Push your branch to GitHub so it appears online
git push origin YOUR-BRANCH-NAME
```

Step 3 — Save Your Work Every Day

After you write code, save it to GitHub with these 3 commands:

```
bash
```

```
git add .
git commit -m "Short description of what you did"
git push origin YOUR-BRANCH-NAME
```

Step 4 — Branch Rules

- **Never push directly to** `main`
 - Each member only works in their own branch
 - Team lead (Aman) merges everything into `main` at the end
 - If you need the latest code from main: `git pull origin main`
-

GitHub Desktop + VS Code Workflow (Pick What Works For You!)

You can choose **any method** below. All methods do the same thing — pick what's easiest for you.

Option A — GitHub Desktop (Easiest — No Commands Needed)

GitHub Desktop is a visual app. No terminal required. Best for beginners.

Setup:

1. Download from desktop.github.com

2. Sign in with your GitHub account
3. Click **File** → **Clone Repository** → paste the repo URL → Clone

Daily workflow in GitHub Desktop:

1. Open GitHub Desktop → select the repo
2. Click **Current Branch** → click **New Branch** → name it (e.g. `aiza/sql-agent`)
3. Write your code in VS Code (File → Open in VS Code)
4. Come back to GitHub Desktop — you'll see your changed files listed
5. Type a summary (e.g. "Added SQL agent logic") → Click **Commit to [branch]**
6. Click **Push origin** (blue button at top)

That's it! GitHub Desktop handles everything visually.

Option B — VS Code Built-in Git (No Extra App Needed)

VS Code has Git built in — no GitHub Desktop needed.

Setup:

1. Open VS Code → File → Open Folder → select the cloned repo folder
2. Click the **Source Control icon** (looks like a fork) on the left sidebar

Daily workflow in VS Code:

1. Bottom-left corner shows your current branch — click it to switch or create a branch
 2. Write your code
 3. Go to Source Control → you'll see changed files
 4. Click + next to each file (or **Stage All Changes**)
 5. Type a commit message → Click the ✓ **checkmark** to commit
 6. Click the ... **menu** → **Push** to send to GitHub
-

Option C — Terminal / Command Line (For Those Who Like Commands)

Full daily workflow:

```
bash
```

```
# Check which branch you're on
git branch

# Switch to your branch
git checkout aiza/sql-agent

# See what files you changed
git status

# Stage all changes
git add .

# Commit with a message
git commit -m "Added SQL controller logic"

# Push to GitHub
git push origin aiza/sql-agent

# Pull latest changes from main (do this if someone else updated main)
git pull origin main
```

Option D — GitHub CLI (Advanced — Optional)

```
bash

# Install GitHub CLI from cli.github.com, then:
gh auth login
gh repo clone aman-ali65/AI-Driven-SQL-Agent

# Create PR from your branch when done
gh pr create --title "Aiza SQL Agent" --body "Completed SQL agent module"
```

Avoiding Merge Conflicts (Important!)

- Only edit files inside **your own module folder**
 - Don't edit `app.py` — that's the team lead's file
 - If you must edit a shared file, tell the team first on WhatsApp/chat
-

requirements.txt (Shared by All)

Note: Some of these are already added in the repo. Check the repo's `requirements.txt` first before reinstalling everything.

```
txt  
  
flask  
flask-cors  
google-generativeai  
openai  
anthropic  
langchain  
langchain-google-genai  
langchain-openai  
langchain-anthropic  
pandas  
openpyxl  
numpy  
PyPDF2  
python-pptx  
python-dotenv  
werkzeug  
matplotlib
```

Install everything:

```
bash  
  
pip install -r requirements.txt
```

Note: `sqlite3` is built into Python — you don't need to install it separately.

Environment Setup (.env file)

Create a `.env` file in the root folder (same folder as `app.py`):

```
env
```

```
# Add whichever keys your team has – you don't need all three
GEMINI_API_KEY=your_gemini_key_here
OPENAI_API_KEY=your_openai_key_here
ANTHROPIC_API_KEY=your_claude_key_here

# The app will use whichever key is available
AI_PROVIDER=gemini    # change to: openai | claude | gemini
```

Where to Get Each API Key

AI	Link	Notes
Gemini (Recommended — Free)	https://aistudio.google.com/api-keys	Free tier available, generous limits
ChatGPT (OpenAI)	https://platform.openai.com/api-keys	Needs billing setup
Claude (Anthropic)	https://console.anthropic.com/settings/keys	Needs billing setup

Recommended: Use Gemini — it's free and works great for this project.

Keep Your Keys Safe!

Add this to your `.gitignore` so you never accidentally upload your keys:

```
.env
uploads/
embeddings/
database/
__pycache__/
*.pyc
```

Main app.py (Team Lead Creates This)

```
python
```

```
from flask import Flask
from flask_cors import CORS

# Import all route blueprints
from modules.sql_agent.agent import sql_bp
from modules.file_to_db.uploader import file_bp
from modules.rag_system.rag import rag_bp
from modules.schema_insights.schema_agent import schema_bp

app = Flask(__name__)
CORS(app) # Allow all origins (needed for frontend testing)

# Register all blueprints (modules)
app.register_blueprint(sql_bp, url_prefix='/sql')
app.register_blueprint(file_bp, url_prefix='/file')
app.register_blueprint(rag_bp, url_prefix='/rag')
app.register_blueprint(schema_bp, url_prefix='/schema')

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

SHARED — Auto Chart / Visualization Module

 Folder: `modules/visualizer/`

 Owner: Aqsa sets it up, everyone calls it

What is the Visualizer?

Whenever a SQL query returns rows of data (a list of dictionaries), the system should **automatically create a chart** so the data is easy to understand.

This is designed to be **very simple** — beginner-friendly. We use **matplotlib**, which is Python's most basic and widely-taught charting library. If you've done any Python data course, you've probably used it.

Why this matters:

- Raw SQL results: `[{"name": "Ali", "score": 85}, {"name": "Sara", "score": 72}]`
- With the visualizer: a bar chart image appears showing scores side by side

The system auto-detects what kind of chart to make:

- 2 columns (one text, one number) → **Bar chart** (most common)
 - 1 column of numbers → **Histogram**
 - More than 2 number columns → **Line chart**
 - If it can't figure it out → just skip the chart, return None
-

✅ Your Complete Code

File: `modules/visualizer/__init__.py` (empty file, just create it)

File: `modules/visualizer/chart_generator.py`

```
python
```

```

# chart_generator.py
# This module automatically generates charts from SQL query results.
# It's designed to be easy to understand – even for beginners!
#
# HOW IT WORKS (simple explanation):
# 1. You pass in a list of dictionaries (SQL results)
# 2. The code looks at what columns exist
# 3. It picks the best chart type automatically
# 4. Saves the chart as a PNG image file
# 5. Returns the file path so the API can send it back

import os
import matplotlib
matplotlib.use('Agg') # Important: use non-interactive mode (no popup windows)
import matplotlib.pyplot as plt
import pandas as pd

# Where to save chart images
CHARTS_FOLDER = "charts"
os.makedirs(CHARTS_FOLDER, exist_ok=True)

def generate_chart(results, query="", filename="chart"):
    """
    Main function – takes SQL results and makes a chart automatically.

    Args:
        results (list): List of dictionaries from SQL query.
                        Example: [{"name": "Ali", "score": 85}, ...]
        query (str): The original SQL query (used for chart title)
        filename (str): Name to save the file as (without extension)

    Returns:
        str: Path to saved chart image, or None if chart couldn't be made.

    EXAMPLE:
        results = [
            {"student": "Ali", "score": 85},
            {"student": "Sara", "score": 72},
            {"student": "Ahmed", "score": 90}
        ]
        path = generate_chart(results)
        # Returns: "charts/chart.png"
    """

    # Step 1: Make sure we have data

```

```

if not results or len(results) == 0:
    return None # No data = no chart

# Step 2: Convert to a pandas DataFrame (easier to work with)
# A DataFrame is like a spreadsheet table in Python
df = pd.DataFrame(results)

# Step 3: Separate columns into text columns and number columns
text_cols = [col for col in df.columns if df[col].dtype == object]
num_cols = [col for col in df.columns if df[col].dtype != object]

# Step 4: Pick the right chart type
chart_path = None

if len(text_cols) == 1 and len(num_cols) == 1:
    # CASE 1: One text column + one number column → Bar Chart
    # Example: name vs score, category vs count
    chart_path = _make_bar_chart(df, text_cols[0], num_cols[0], filename, query)

elif len(num_cols) == 1 and len(text_cols) == 0:
    # CASE 2: Only one number column → Histogram
    # Example: just a list of scores
    chart_path = _make_histogram(df, num_cols[0], filename, query)

elif len(num_cols) >= 2:
    # CASE 3: Multiple number columns → Line Chart
    # Example: scores and grades together
    chart_path = _make_line_chart(df, num_cols, filename, query)

# If none of the cases matched, return None (no chart)
return chart_path

def _make_bar_chart(df, x_col, y_col, filename, title):
    """
    Makes a bar chart. Best for comparing categories.
    Example: student names on X-axis, scores on Y-axis.
    """
    fig, ax = plt.subplots(figsize=(10, 6)) # Create a 10x6 inch figure

    # Draw the bars
    ax.bar(df[x_col].astype(str), df[y_col], color='steelblue', edgecolor='white')

    # Add labels
    ax.set_xlabel(x_col.replace('_', ' ').title(), fontsize=12)
    ax.set_ylabel(y_col.replace('_', ' ').title(), fontsize=12)
    ax.set_title(title if title else f"{y_col} by {x_col}", fontsize=14)

```

```

# Rotate x-axis labels if there are many bars (so they don't overlap)
if len(df) > 5:
    plt.xticks(rotation=45, ha='right')

plt.tight_layout() # Adjust spacing

# Save the chart
path = os.path.join(CHARTS_FOLDER, f"{filename}.png")
plt.savefig(path, dpi=100, bbox_inches='tight')
plt.close(fig) # Close to free memory

return path

def _make_histogram(df, col, filename, title):
    """
    Makes a histogram. Best for showing distribution of one number.
    Example: how scores are spread (most students got 70-80?).
    """
    fig, ax = plt.subplots(figsize=(10, 6))

    ax.hist(df[col].dropna(), bins=10, color='coral', edgecolor='white')
    ax.set_xlabel(col.replace('_', ' ').title(), fontsize=12)
    ax.set_ylabel('Count', fontsize=12)
    ax.set_title(title if title else f"Distribution of {col}", fontsize=14)

    plt.tight_layout()
    path = os.path.join(CHARTS_FOLDER, f"{filename}.png")
    plt.savefig(path, dpi=100, bbox_inches='tight')
    plt.close(fig)

    return path

def _make_line_chart(df, num_cols, filename, title):
    """
    Makes a line chart. Best for showing trends across multiple columns.
    """
    fig, ax = plt.subplots(figsize=(10, 6))

    for col in num_cols:
        ax.plot(df.index, df[col], marker='o', label=col.replace('_', ' ').title())

    ax.set_xlabel('Row Index', fontsize=12)
    ax.set_ylabel('Values', fontsize=12)
    ax.set_title(title if title else "Trend Chart", fontsize=14)

```

```
ax.legend()

plt.tight_layout()
path = os.path.join(CHARTS_FOLDER, f"{filename}.png")
plt.savefig(path, dpi=100, bbox_inches='tight')
plt.close(fig)

return path
```

How Other Modules Use This

Any module can call the chart generator after getting SQL results:

```
python

# Inside agent.py or schema_agent.py, after executing SQL:
from modules.visualizer.chart_generator import generate_chart

result = execute_sql(generated_sql)

chart_path = None
if result["success"] and result["data"]:
    chart_path = generate_chart(
        results=result["data"],
        query=generated_sql,
        filename="query_result"
    )

return jsonify({
    "status": "success",
    "generated_sql": generated_sql,
    "result": result,
    "chart": chart_path # Will be None if no chart was made
})
```

Note for beginners: matplotlib saves images to disk. In a real app, you'd serve these as static files or convert to base64. For now, returning the file path is enough for the project.



AIZA — SQL Agent Module



Branch Name: `aiza/sql-agent`



Folder: `modules/sql_agent/`



Deadline: Saturday 2nd May, 12:00 PM

What is the SQL Agent?

The SQL Agent is the brain of the project. When a user types a question like:

┆ "Show me all students who scored above 80"

The SQL Agent:

1. Takes that question
 2. Reads the database schema (table names + columns) from SQLite
 3. Sends the schema + question to the AI (GPT, Gemini, or Claude — whichever key is available)
 4. The AI returns a SQL query like: `SELECT * FROM students WHERE score > 80`
 5. The validator checks if the query is safe to run
 6. If `auto_execute = false` → returns the SQL for human approval before running
 7. If `auto_execute = true` → runs the query directly and returns results + chart
-

Step-by-Step: How It Works

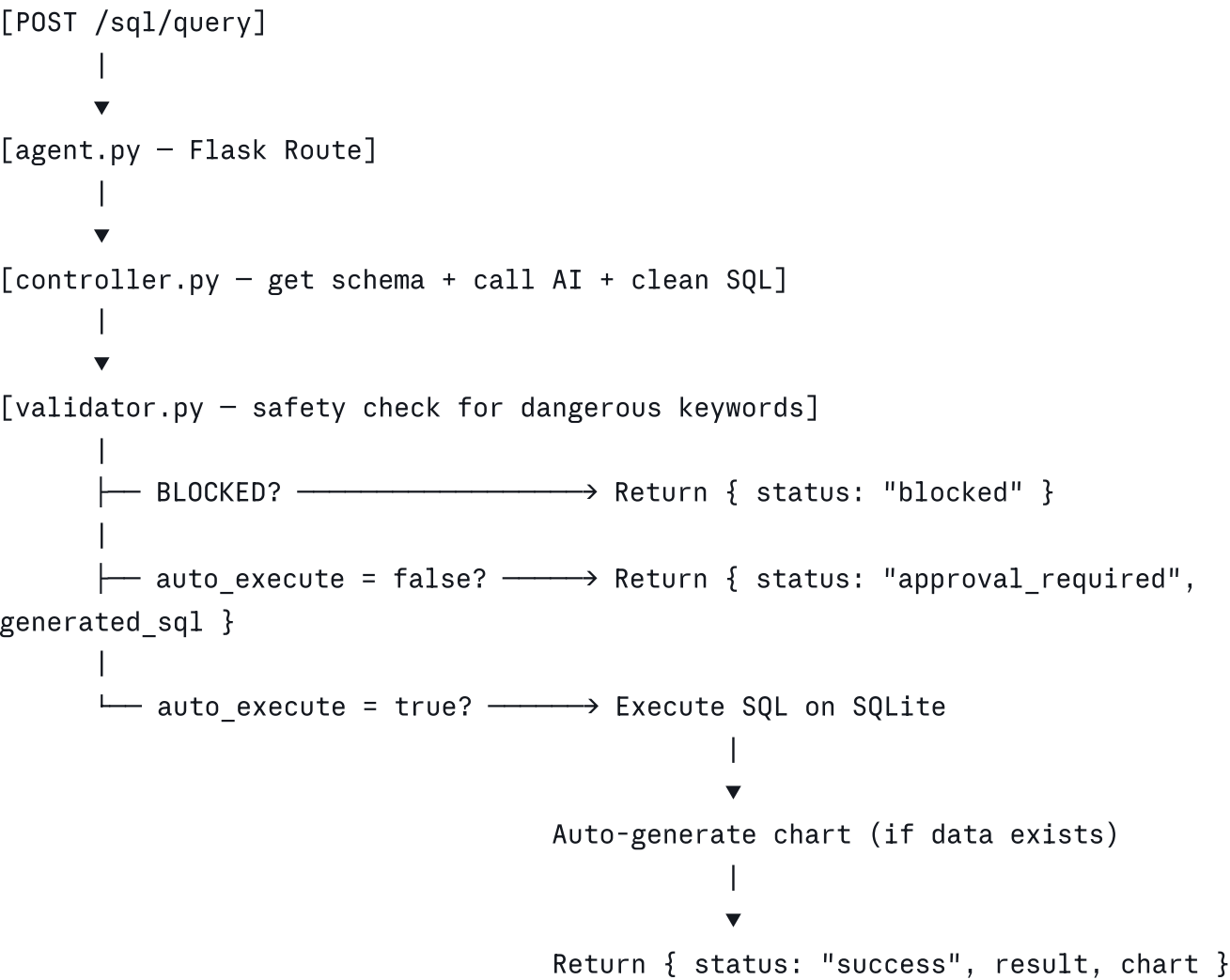
```
User sends question + auto_execute flag
    ↓
controller.py gets the DB schema (table names + columns)
    ↓
Sends: [Schema + User Question] → AI (GPT / Gemini / Claude)
    ↓
AI returns: SQL query (e.g., SELECT * FROM students WHERE...)
    ↓
controller.py cleans the SQL (removes backticks, extra spaces)
    ↓
validator.py checks for dangerous keywords (DROP, DELETE, TRUNCATE, ALTER)
    ↓
[If blocked] → return status: "blocked"
[If auto_execute = false] → return SQL for approval (status:
"approval_required")
```

[If auto_execute = true] → execute SQL on SQLite → generate chart → return results

↓

Returns structured JSON response

Architecture Diagram (Simple)



Key Components

Component	File	What it does
get_schema()	controller.py	Reads all table names and columns from SQLite
generate_sql()	controller.py	Sends question + schema to AI (GPT/Gemini/Claude), gets SQL back
clean_sql()	controller.py	Removes any markdown formatting from AI output
execute_sql()	controller.py	Runs the SQL query safely on SQLite

Component	File	What it does
<code>is_dangerous()</code>	<code>validator.py</code>	Checks if SQL contains DROP, DELETE, TRUNCATE, ALTER
Flask Route <code>/sql/query</code>	<code>agent.py</code>	API endpoint: takes question + auto_execute, returns structured JSON
Flask Route <code>/sql/schema</code>	<code>agent.py</code>	Returns current DB schema as text

 Your Complete Code

File: `modules/sql_agent/__init__.py` (empty file, just create it)

File: `modules/sql_agent/validator.py`

```
python
```



```
# validator.py
# This file is responsible for checking if a SQL query is safe to run.
# It blocks dangerous operations like DROP, DELETE, TRUNCATE, ALTER
# unless the user has explicitly approved them.
```

```
DANGEROUS_KEYWORDS = ["DROP", "DELETE", "TRUNCATE", "ALTER"]
```

```
def is_dangerous(sql):
    """
    Check if the SQL query contains any dangerous keywords.

    Args:
        sql (str): The SQL query to check.

    Returns:
        bool: True if dangerous, False if safe.

    Example:
        is_dangerous("DROP TABLE students") → True
        is_dangerous("SELECT * FROM students") → False
    """
    upper_sql = sql.upper()

    for keyword in DANGEROUS_KEYWORDS:
        if keyword in upper_sql:
            return True

    return False
```

File: `modules/sql_agent/controller.py`

```
python
```

```
# controller.py
# This file handles the core LLM + database logic.
# It supports GPT (OpenAI), Gemini (Google), and Claude (Anthropic).
# The AI used depends on which API key is set in the .env file.

import os
import sqlite3
from dotenv import load_dotenv

load_dotenv()

DB_PATH = "database/data.db"
AI_PROVIDER = os.getenv("AI_PROVIDER", "gemini").lower() # Default: gemini

# — Load the right AI client based on .env setting —————

def _get_sql_from_gemini(prompt):
    import google.generativeai as genai
    genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
    model = genai.GenerativeModel("gemini-pro")
    response = model.generate_content(prompt)
    return response.text

def _get_sql_from_openai(prompt):
    from openai import OpenAI
    client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
    response = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}],
        temperature=0
    )
    return response.choices[0].message.content

def _get_sql_from_claude(prompt):
    import anthropic
    client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))
    message = client.messages.create(
        model="claude-3-haiku-20240307",
        max_tokens=500,
        messages=[{"role": "user", "content": prompt}]
    )
    return message.content[0].text
```

```

def call_ai(prompt):
    """
    Sends a prompt to whichever AI is configured in .env.
    Supports: gemini, openai (GPT), claude (Anthropic)
    """
    if AI_PROVIDER == "openai":
        return _get_sql_from_openai(prompt)
    elif AI_PROVIDER == "claude":
        return _get_sql_from_claude(prompt)
    else:
        return _get_sql_from_gemini(prompt) # Default

# — Database functions —————

def get_schema():
    """
    Reads all table names and their columns from SQLite.

    Returns:
        str: A formatted string like:
            Table: students | Columns: id, name, score, grade
    """
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
    tables = cursor.fetchall()

    schema_text = ""
    for (table_name,) in tables:
        cursor.execute(f"PRAGMA table_info({table_name});")
        columns = cursor.fetchall()
        col_names = [col[1] for col in columns]
        schema_text += f"Table: {table_name} | Columns: {'', ' '.join(col_names)}\n"

    conn.close()
    return schema_text

def clean_sql(raw_sql):
    """
    AI sometimes wraps SQL in markdown. This removes that formatting.
    """
    sql = raw_sql.strip()
    sql = sql.replace("`sql", "").replace("`", "")

```

```
return sql.strip()
```

```
def generate_sql(user_question, schema):
```

```
    """
```

```
    Sends the schema + user question to the AI.
```

```
    Returns the generated SQL query.
```

```
    """
```

```
    prompt = f"""You are an expert SQL assistant. Given the database schema below, w
```

DATABASE SCHEMA:

{schema}

USER QUESTION:

{user_question}

IMPORTANT RULES:

- Return ONLY the SQL query, nothing else
- Do NOT add any explanation
- Do NOT use markdown formatting
- Make sure the SQL is valid for SQLite

SQL QUERY:

```
    """
```

```
    raw_sql = call_ai(prompt)
```

```
    return clean_sql(raw_sql)
```

```
def execute_sql(sql_query):
```

```
    """
```

```
    Runs the SQL query on the SQLite database.
```

```
    Returns:
```

```
        dict: { "success": True, "data": [...], "count": N }
```

```
    """
```

```
    conn = sqlite3.connect(DB_PATH)
```

```
    cursor = conn.cursor()
```

```
    try:
```

```
        cursor.execute(sql_query)
```

```
        rows = cursor.fetchall()
```

```
        col_names = [desc[0] for desc in cursor.description] if cursor.description e
```

```
        results = [dict(zip(col_names, row)) for row in rows]
```

```
        conn.close()
```

```
        return {"success": True, "data": results, "count": len(results)}
```

```
    except Exception as e:
```

```
conn.close()  
return {"success": False, "error": str(e)}
```

File: `modules/sql_agent/agent.py`

python

```

# agent.py
# Flask API routes for the SQL Agent module.

from flask import Blueprint, request, jsonify
from modules.sql_agent.controller import get_schema, generate_sql, execute_sql
from modules.sql_agent.validator import is_dangerous
from modules.visualizer.chart_generator import generate_chart

sql_bp = Blueprint('sql', __name__)

@sql_bp.route('/query', methods=['POST'])
def query():
    """
    POST /sql/query

    Request Body:
    {
        "question": "Show students with marks above 80",
        "auto_execute": false
    }
    """
    data = request.get_json()

    if not data or 'question' not in data:
        return jsonify({"error": "Please provide a 'question' in the request body"})

    user_question = data['question']
    auto_execute = data.get('auto_execute', False)

    schema = get_schema()
    if not schema:
        return jsonify({"error": "No tables found. Please upload a file first."}), 404

    generated_sql = generate_sql(user_question, schema)

    if is_dangerous(generated_sql):
        return jsonify({
            "status": "blocked",
            "generated_sql": generated_sql,
            "explanation": "This query contains a dangerous operation and was blocked"
        })

    if not auto_execute:
        return jsonify({
            "status": "approval_required",

```

```

        "generated_sql": generated_sql,
        "explanation": "Review the SQL. Send with 'auto_execute': true to run it
    })

    result = execute_sql(generated_sql)

    # Auto-generate chart if results contain data
    chart_path = None
    if result["success"] and result["data"]:
        chart_path = generate_chart(
            results=result["data"],
            query=generated_sql,
            filename="sql_result"
        )

    return jsonify({
        "status": "success",
        "generated_sql": generated_sql,
        "result": result,
        "chart": chart_path
    })

@sql_bp.route('/schema', methods=['GET'])
def show_schema():
    """
    GET /sql/schema
    Returns the current database schema as text.
    """
    schema = get_schema()
    return jsonify({"schema": schema})

```

How to Test Your Module

Use **Postman** or `curl`:

```
bash
```

```
# Step 1: Ask a question – preview the SQL first (auto_execute: false)
curl -X POST http://localhost:5000/sql/query \
  -H "Content-Type: application/json" \
  -d '{"question": "Show students with marks above 80", "auto_execute": false}'

# Step 2: Approve and run the query (auto_execute: true)
curl -X POST http://localhost:5000/sql/query \
  -H "Content-Type: application/json" \
  -d '{"question": "Show students with marks above 80", "auto_execute": true}'

# Test a dangerous query – should be blocked
curl -X POST http://localhost:5000/sql/query \
  -H "Content-Type: application/json" \
  -d '{"question": "Delete all students", "auto_execute": true}'

# View current DB schema
curl http://localhost:5000/sql/schema
```

Challenges & Solutions

Challenge	Solution
AI returns SQL with backticks	Use <code>clean_sql()</code> to strip markdown
User runs a dangerous query	<code>validator.py</code> blocks it — returns <code>status: blocked</code>
Query returns no results	Check schema first using <code>GET /sql/schema</code>
Wrong API key / AI not working	Check your <code>.env</code> file and <code>AI_PROVIDER</code> setting
Database file not found	Make sure a file was uploaded via Habiba's module first

HABIBA — File to Database Module

 **Branch Name:** `habiba/file-to-db`

 **Folder:** `modules/file_to_db/`

 **Deadline:** Saturday 2nd May, 12:00 PM

What is the File to Database Module?

This module lets users upload a CSV or Excel file. The system automatically:

1. Reads the file using pandas
2. Creates a table in SQLite with the correct column names and types
3. Inserts all the data into the table

Example: User uploads `students.csv` with columns: `name, age, score` → System creates a SQLite table called `students` → All rows from the CSV are inserted into the table → Now Aiza's SQL Agent can query this table!

Step-by-Step: How It Works

```
User uploads CSV/Excel file
      ↓
Save file to /uploads/ folder
      ↓
Read file using pandas (pd.read_csv or pd.read_excel)
      ↓
Clean column names (remove spaces, lowercase)
      ↓
Create SQLite table using pandas .to_sql()
      ↓
Return success message + table name + preview
```

Key Components

Component	What it does
<code>allowed_file()</code>	Checks if uploaded file is CSV or Excel
<code>clean_column_names()</code>	Removes spaces and special characters from column names
<code>read_file()</code>	Reads CSV or Excel into a pandas DataFrame
<code>save_to_db()</code>	Saves the DataFrame as a SQLite table
Flask Route <code>/file/upload</code>	API endpoint that accepts file upload

✔ Your Complete Code

File: `modules/file_to_db/__init__.py` (empty file, just create it)

File: `modules/file_to_db/uploader.py`

```
python
```

```

import os
import sqlite3
import pandas as pd
from flask import Blueprint, request, jsonify
from werkzeug.utils import secure_filename

file_bp = Blueprint('file', __name__)

UPLOAD_FOLDER = "uploads"
DB_PATH = "database/data.db"

os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs("database", exist_ok=True)

ALLOWED_EXTENSIONS = {'csv', 'xlsx', 'xls'}

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

def clean_column_names(df):
    """
    Clean column names so they work in SQL.
    Example: "Student Name" → "student_name"
    """
    df.columns = (
        df.columns
        .str.strip()
        .str.lower()
        .str.replace(' ', '_')
        .str.replace(r'[^\\w]', '_', regex=True)
    )
    return df

def read_file(filepath, extension):
    if extension == 'csv':
        df = pd.read_csv(filepath)
    elif extension in ['xlsx', 'xls']:
        df = pd.read_excel(filepath)
    else:
        raise ValueError(f"Unsupported file type: {extension}")
    return df

```

```

def save_to_db(df, table_name):
    conn = sqlite3.connect(DB_PATH)
    df.to_sql(table_name, conn, if_exists='replace', index=False)
    conn.close()

@file_bp.route('/upload', methods=['POST'])
def upload_file():
    """
    POST /file/upload
    Form-data: file = [your CSV/Excel file]
    Returns: table name + column names + preview rows
    """
    if 'file' not in request.files:
        return jsonify({"error": "No file found. Use key 'file'."}), 400

    file = request.files['file']

    if file.filename == '':
        return jsonify({"error": "No file selected"}), 400

    if not allowed_file(file.filename):
        return jsonify({"error": "Only CSV, XLSX, or XLS files are allowed"}), 400

    filename = secure_filename(file.filename)
    filepath = os.path.join(UPLOAD_FOLDER, filename)
    file.save(filepath)

    extension = filename.rsplit('.', 1)[1].lower()

    try:
        df = read_file(filepath, extension)
        df = clean_column_names(df)

        table_name = filename.rsplit('.', 1)[0].lower().replace('-', '_').replace(' ', '_')
        save_to_db(df, table_name)

        preview = df.head(5).to_dict(orient='records')

        return jsonify({
            "success": True,
            "message": f"File uploaded and saved as table '{table_name}'",
            "table_name": table_name,
            "columns": list(df.columns),
            "total_rows": len(df),
            "preview": preview
        })

```

```

except Exception as e:
    return jsonify({"error": f"Failed to process file: {str(e)}"}), 500

@file_bp.route('/tables', methods=['GET'])
def list_tables():
    """
    GET /file/tables
    Returns: list of all tables in the database
    """
    try:
        conn = sqlite3.connect(DB_PATH)
        cursor = conn.cursor()
        cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
        tables = [row[0] for row in cursor.fetchall()]
        conn.close()
        return jsonify({"tables": tables})
    except Exception as e:
        return jsonify({"error": str(e)}), 500

```

How to Test Your Module

```

bash

# Upload a CSV file (use Postman → form-data → key: file → select your file)
curl -X POST http://localhost:5000/file/upload \
  -F "file=@students.csv"

# List all tables
curl http://localhost:5000/file/tables

```

Test CSV (save as `students.csv`):

```

csv

Name, Age, Score, Grade
Ali, 20, 85, A
Sara, 21, 72, B
Ahmed, 19, 90, A+
Zara, 22, 65, C

```

Challenges & Solutions

Challenge	Solution
Column names have spaces	Use <code>clean_column_names()</code> to replace spaces with underscores
Excel file has multiple sheets	Pandas reads the first sheet by default
File already exists in DB	Use <code>if_exists='replace'</code> in <code>to_sql()</code>
Non-UTF characters in CSV	Add <code>encoding='utf-8-sig'</code> in <code>read_csv()</code> if error occurs



REHAN — RAG System (Document AI) Module



Branch Name: `rehan/rag-system`



Folder: `modules/rag_system/`



Deadline: Saturday 2nd May, 12:00 PM

What is RAG?

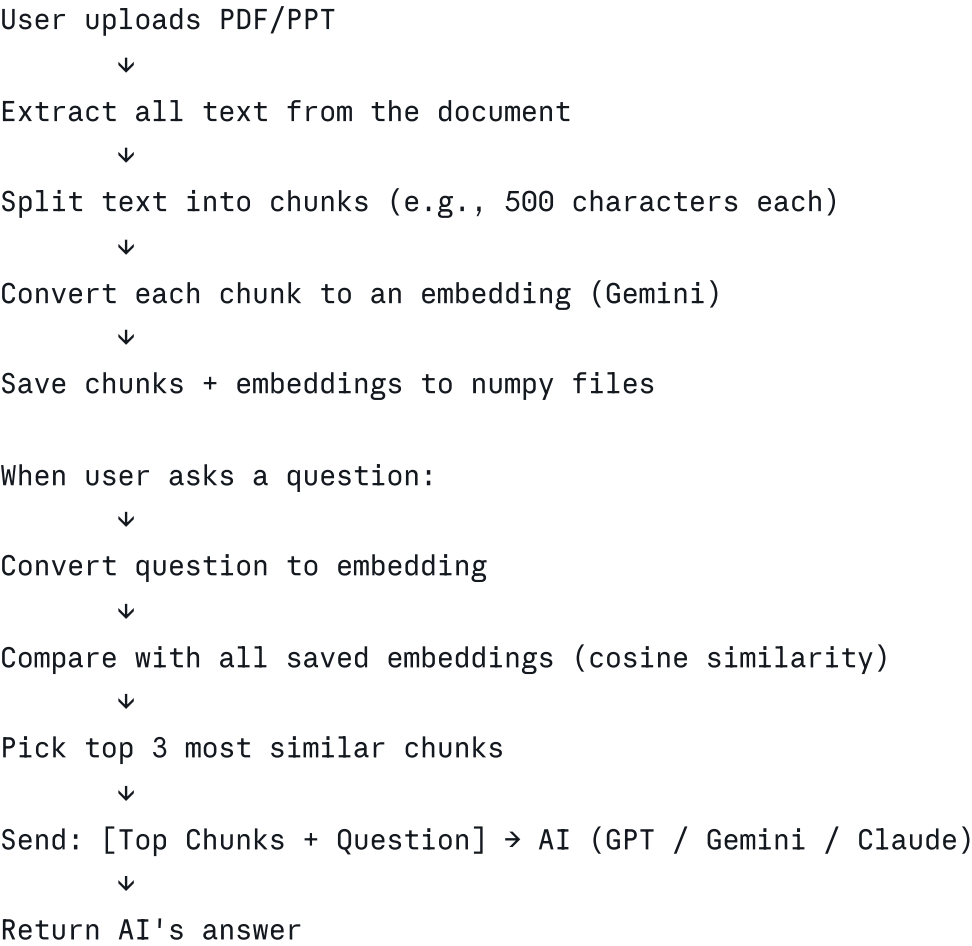
RAG stands for **Retrieval Augmented Generation**.

In simple words:

1. User uploads a PDF or PPT
2. The system reads and breaks it into small pieces (chunks)
3. Each chunk is converted into a list of numbers (embedding)
4. When user asks a question, the question is also converted to numbers
5. The system finds the chunks most similar to the question
6. Sends those chunks to the AI (GPT, Gemini, or Claude)
7. The AI reads those chunks and answers the question

Why not just send the whole document to AI? Because documents can be huge. Embeddings let us find only the relevant parts!

Step-by-Step: How It Works



Key Components

Component	What it does
<code>extract_text_from_pdf()</code>	Reads all text from a PDF
<code>extract_text_from_ppt()</code>	Reads all text from a PPT
<code>chunk_text()</code>	Splits text into small pieces
<code>get_embedding()</code>	Converts text to a list of numbers using Gemini
<code>cosine_similarity()</code>	Compares two embeddings, returns score
<code>find_top_chunks()</code>	Finds the 3 most relevant chunks for a question
<code>answer_question()</code>	Sends relevant chunks to AI, returns answer
<code>/rag/upload</code>	Upload a document
<code>/rag/query</code>	Ask a question

 Your Complete Code

File: `modules/rag_system/__init__.py` (empty file)

File: `modules/rag_system/rag.py`

```
python
```



```
import os
import numpy as np
import json
from flask import Blueprint, request, jsonify
from werkzeug.utils import secure_filename
import google.generativeai as genai
from dotenv import load_dotenv

import PyPDF2
from pptx import Presentation

load_dotenv()

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel("gemini-pro")

rag_bp = Blueprint('rag', __name__)

UPLOAD_FOLDER = "uploads"
EMBEDDINGS_FOLDER = "embeddings"

os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(EMBEDDINGS_FOLDER, exist_ok=True)

ALLOWED_DOC_EXTENSIONS = {'pdf', 'pptx', 'ppt'}

def allowed_doc(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_DOC_EXTENSIONS

def extract_text_from_pdf(filepath):
    text = ""
    with open(filepath, 'rb') as f:
        reader = PyPDF2.PdfReader(f)
        for page in reader.pages:
            page_text = page.extract_text()
            if page_text:
                text += page_text + "\n"
    return text

def extract_text_from_ppt(filepath):
    text = ""
    prs = Presentation(filepath)
    for slide in prs.slides:
```

```

        for shape in slide.shapes:
            if hasattr(shape, "text"):
                text += shape.text + "\n"
    return text

def chunk_text(text, chunk_size=500, overlap=50):
    """
    Split a large text into smaller pieces.
    chunk_size = how many characters per chunk
    overlap = how many characters repeat between chunks (for context)
    """
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end]
        if chunk.strip():
            chunks.append(chunk)
            start = end - overlap
    return chunks

def get_embedding(text):
    """
    Convert text into a list of numbers (embedding) using Gemini.
    These numbers represent the meaning of the text.
    """
    result = genai.embed_content(
        model="models/embedding-001",
        content=text,
        task_type="retrieval_document"
    )
    return result['embedding']

def cosine_similarity(vec1, vec2):
    """
    Compare two embeddings.
    Returns a score between 0 and 1. Closer to 1 = more similar.
    """
    vec1 = np.array(vec1)
    vec2 = np.array(vec2)
    dot_product = np.dot(vec1, vec2)
    magnitude = np.linalg.norm(vec1) * np.linalg.norm(vec2)
    if magnitude == 0:
        return 0

```

```

    return dot_product / magnitude

def find_top_chunks(question_embedding, all_embeddings, all_chunks, top_k=3):
    scores = []
    for i, chunk_embedding in enumerate(all_embeddings):
        score = cosine_similarity(question_embedding, chunk_embedding)
        scores.append((score, i))
    scores.sort(reverse=True)
    return [all_chunks[i] for score, i in scores[:top_k]]

def answer_with_ai(question, relevant_chunks):
    """
    Send the relevant document chunks + question to Gemini.
    """
    context = "\n\n---\n\n".join(relevant_chunks)
    prompt = f"""You are a helpful AI assistant. Use ONLY the information in the context to answer the question. If the answer is not in the context, say "I could not find this information in the context".

CONTEXT:
{context}

QUESTION:
{question}

ANSWER:
"""
    response = model.generate_content(prompt)
    return response.text

@rag_bp.route('/upload', methods=['POST'])
def upload_document():
    """
    POST /rag/upload
    Form-data: file = [your PDF or PPTX]
    """
    if 'file' not in request.files:
        return jsonify({"error": "No file in request"}), 400

    file = request.files['file']
    if not allowed_doc(file.filename):
        return jsonify({"error": "Only PDF, PPTX files allowed"}), 400

    filename = secure_filename(file.filename)
    filepath = os.path.join(UPLOAD_FOLDER, filename)

```

```

file.save(filepath)

extension = filename.rsplit('.', 1)[1].lower()

try:
    if extension == 'pdf':
        text = extract_text_from_pdf(filepath)
    else:
        text = extract_text_from_ppt(filepath)

    if not text.strip():
        return jsonify({"error": "Could not extract text from file"}), 400

    chunks = chunk_text(text)

    embeddings = [get_embedding(chunk) for chunk in chunks]

    doc_name = filename.rsplit('.', 1)[0]
    np.save(os.path.join(EMBEDDINGS_FOLDER, f"{doc_name}_embeddings.npy"), np.array(embeddings))

    with open(os.path.join(EMBEDDINGS_FOLDER, f"{doc_name}_chunks.json"), 'w') as f:
        json.dump(chunks, f)

    return jsonify({
        "success": True,
        "document": doc_name,
        "chunks_created": len(chunks),
        "message": f"Document processed. {len(chunks)} chunks created."
    })

except Exception as e:
    return jsonify({"error": str(e)}), 500

@rag_bp.route('/query', methods=['POST'])
def query_document():
    """
    POST /rag/query
    Body: { "question": "What is the main topic?", "document": "lecture_notes" }
    """
    data = request.get_json()
    if not data or 'question' not in data or 'document' not in data:
        return jsonify({"error": "Provide 'question' and 'document' name"}), 400

    question = data['question']
    doc_name = data['document']

```

```

emb_path = os.path.join(EMBEDDINGS_FOLDER, f"{doc_name}_embeddings.npy")
chunks_path = os.path.join(EMBEDDINGS_FOLDER, f"{doc_name}_chunks.json")

if not os.path.exists(emb_path) or not os.path.exists(chunks_path):
    return jsonify({"error": f"Document '{doc_name}' not found. Upload it first."})

try:
    all_embeddings = np.load(emb_path)
    with open(chunks_path, 'r') as f:
        all_chunks = json.load(f)

    question_embedding = get_embedding(question)
    top_chunks = find_top_chunks(question_embedding, all_embeddings, all_chunks)
    answer = answer_with_ai(question, top_chunks)

    return jsonify({
        "question": question,
        "answer": answer,
        "source_chunks": top_chunks
    })

except Exception as e:
    return jsonify({"error": str(e)}), 500

```

How to Test Your Module

```

bash

# Upload a PDF
curl -X POST http://localhost:5000/rag/upload -F "file=@lecture.pdf"

# Query the document
curl -X POST http://localhost:5000/rag/query \
  -H "Content-Type: application/json" \
  -d '{"question": "What is machine learning?", "document": "lecture"}'

```

Challenges & Solutions

Challenge	Solution
PDF has scanned images (no text)	PyPDF2 can't read images — note this as a limitation
Too many API calls for embedding	500-char chunks is a good size — don't make them smaller

Challenge	Solution
Embeddings take long to generate	Normal — embeddings are expensive, done once at upload
Answer doesn't match document	Increase <code>top_k</code> from 3 to 5 to get more context

AQSA — Schema Agent + Insights Module

 **Branch Name:** `aqsa/schema-insights`

 **Folder:** `modules/schema_insights/`

 **Deadline:** Saturday 2nd May, 12:00 PM

What is the Schema Agent?

The Schema Agent lets users manage the database structure using plain English.

Examples:

- "Create a table called employees with name, age, salary" → AI creates it
- "Add a column email to employees" → AI generates `ALTER TABLE`
- "Delete the test table" → AI generates `DROP TABLE`

What is the Insights Engine?

After the SQL Agent gets results, the Insights Engine analyzes them and tells the user something useful.

Example:

- SQL Result: `[{score: 85}, {score: 72}, {score: 90}]`
- Insight: "The average score is 82.3. The highest score is 90 and the lowest is 72."

Your Additional Task — Set Up the Visualizer Module

Since you're working on insights, you also set up the **shared visualizer** module (see the Visualizer section above). Create the folder and file — it's straightforward code already written for you.

Step-by-Step: How Schema Agent Works

User: "Create a table called products with name, price, stock"

↓

Send to AI (GPT / Gemini / Claude): "Generate a CREATE TABLE SQL for: [user request]"

↓

AI returns: CREATE TABLE products (name TEXT, price REAL, stock INTEGER);

↓

Show SQL to user (HITL – Human in the Loop)

↓

User confirms → Execute the SQL → Return success

Key Components

Component	What it does
<code>generate_schema_sql()</code>	Uses AI to generate CREATE/ALTER/DROP SQL
<code>execute_schema_sql()</code>	Runs DDL queries safely on SQLite
<code>generate_insights()</code>	Sends results to AI for analysis
<code>detect_query_type()</code>	Checks if SQL is dangerous (DROP, DELETE)
<code>/schema/modify</code>	API to create/alter/drop tables
<code>/schema/show</code>	View all table schemas
<code>/schema/insights</code>	Get AI insights from query results

✅ Your Complete Code

File: `modules/schema_insights/__init__.py` (empty file)

File: `modules/schema_insights/schema_agent.py`

python

```

import sqlite3
import os
from flask import Blueprint, request, jsonify
import google.generativeai as genai
from dotenv import load_dotenv
from modules.visualizer.chart_generator import generate_chart

load_dotenv()

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel("gemini-pro")

schema_bp = Blueprint('schema', __name__)

DB_PATH = "database/data.db"
os.makedirs("database", exist_ok=True)

def generate_schema_sql(user_request):
    """
    Send user's plain English schema request to AI.
    AI returns a CREATE TABLE / ALTER TABLE / DROP TABLE SQL.
    """
    prompt = f"""
You are a SQLite database expert. Convert the user's request into a valid SQLite DDL

USER REQUEST:
{user_request}

IMPORTANT RULES:
- Only generate CREATE TABLE, ALTER TABLE, or DROP TABLE statements
- Use TEXT, INTEGER, or REAL as column types (SQLite types)
- Return ONLY the SQL, no explanation, no markdown
- Make sure SQL is valid for SQLite

SQL:
"""
    response = model.generate_content(prompt)
    raw = response.text.strip()
    raw = raw.replace("`sql", "").replace("`", "").strip()
    return raw

def detect_dangerous_query(sql):
    upper_sql = sql.upper()
    dangerous_keywords = ["DROP", "DELETE", "TRUNCATE"]

```



```

for keyword in dangerous_keywords:
    if keyword in upper_sql:
        return True
return False

def execute_schema_sql(sql):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    try:
        cursor.execute(sql)
        conn.commit()
        conn.close()
        return {"success": True, "message": "SQL executed successfully"}
    except Exception as e:
        conn.close()
        return {"success": False, "error": str(e)}

def get_full_schema():
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
    tables = cursor.fetchall()

    schema_info = {}
    for (table_name,) in tables:
        cursor.execute(f"PRAGMA table_info({table_name});")
        columns = cursor.fetchall()
        schema_info[table_name] = [
            {"column": col[1], "type": col[2], "nullable": not col[3]}
            for col in columns
        ]

    conn.close()
    return schema_info

def generate_insights(results, query=""):
    if not results:
        return "No data to analyze."

    if isinstance(results, list) and len(results) > 0:
        columns = list(results[0].keys())
        rows_text = "\n".join([str(row) for row in results[:20]])
        data_summary = f"Columns: {', '.join(columns)}\nData (first {min(20, len(res"
    else:

```

```
data_summary = str(results)
```

```
prompt = f"""
```

You are a data analyst. Analyze the following data and provide useful insights.

```
ORIGINAL QUERY (if available): {query}
```

```
DATA:
```

```
{data_summary}
```

```
Provide:
```

1. A brief summary of what the data shows
2. Key observations (min, max, averages if applicable)
3. Any interesting patterns or anomalies

Keep it short and easy to understand. Speak like a data analyst explaining to a student.

```
INSIGHTS:
```

```
"""
```

```
response = model.generate_content(prompt)
```

```
return response.text
```

```
@schema_bp.route('/modify', methods=['POST'])
```

```
def modify_schema():
```

```
    """
```

```
    POST /schema/modify
```

```
    Body: {
```

```
        "request": "Create a table called employees with name, age, department",
```

```
        "confirm": false
```

```
    }
```

```
    """
```

```
    data = request.get_json()
```

```
    if not data or 'request' not in data:
```

```
        return jsonify({"error": "Provide a 'request' describing what to do"}), 400
```

```
    user_request = data['request']
```

```
    confirm = data.get('confirm', False)
```

```
    sql = generate_schema_sql(user_request)
```

```
    is_dangerous = detect_dangerous_query(sql)
```

```
    if not confirm:
```

```
        return jsonify({
```

```
            "generated_sql": sql,
```

```
            "is_dangerous": is_dangerous,
```

```
        "message": "Review the SQL. Send with 'confirm': true to execute.",
        "warning": "⚠ This is a destructive operation! Be careful." if is_dangerous else ""
    })
```

```
result = execute_schema_sql(sql)
result["generated_sql"] = sql
return jsonify(result)
```

```
@schema_bp.route('/show', methods=['GET'])
```

```
def show_schema():
```

```
    """
```

```
    GET /schema/show
```

```
    Returns full schema with all tables and column details.
```

```
    """
```

```
    schema = get_full_schema()
```

```
    return jsonify({
```

```
        "tables": schema,
```

```
        "table_count": len(schema)
```

```
    })
```

```
@schema_bp.route('/insights', methods=['POST'])
```

```
def get_insights():
```

```
    """
```

```
    POST /schema/insights
```

```
    Body: { "results": [...], "query": "original SQL query" }
```

```
    Returns AI-generated insights + auto chart.
```

```
    """
```

```
    data = request.get_json()
```

```
    if not data or 'results' not in data:
```

```
        return jsonify({"error": "Provide 'results' (list of rows from SQL query)"})
```

```
    results = data['results']
```

```
    query = data.get('query', '')
```

```
    if not results:
```

```
        return jsonify({"insights": "No data provided to analyze."})
```

```
    insights = generate_insights(results, query)
```

```
    # Auto-generate chart from the results
```

```
    chart_path = generate_chart(
```

```
        results=results,
```

```
        query=query,
```

```
        filename="insights_chart"
```

```

)

return jsonify({
    "insights": insights,
    "rows_analyzed": len(results),
    "chart": chart_path
})

@schema_bp.route('/drop', methods=['POST'])
def drop_table():
    """
    POST /schema/drop
    Body: { "table_name": "old_table", "confirm": true }
    """
    data = request.get_json()

    if not data or 'table_name' not in data:
        return jsonify({"error": "Provide 'table_name'"}), 400

    table_name = data['table_name']
    confirm = data.get('confirm', False)

    sql = f"DROP TABLE IF EXISTS {table_name};"

    if not confirm:
        return jsonify({
            "generated_sql": sql,
            "message": "Send 'confirm': true to permanently delete this table.",
            "warning": f"⚠ This will permanently delete table '{table_name}' and all"
        })

    result = execute_schema_sql(sql)
    result["dropped_table"] = table_name
    return jsonify(result)

```

How to Test Your Module

```
bash
```

```
# Preview a schema change (confirm: false)
curl -X POST http://localhost:5000/schema/modify \
  -H "Content-Type: application/json" \
  -d '{"request": "Create a table employees with columns name, age, salary", "confirm": false}'

# Execute the schema change (confirm: true)
curl -X POST http://localhost:5000/schema/modify \
  -H "Content-Type: application/json" \
  -d '{"request": "Create a table employees with columns name, age, salary", "confirm": true}'

# Show all tables
curl http://localhost:5000/schema/show

# Get insights + chart from results
curl -X POST http://localhost:5000/schema/insights \
  -H "Content-Type: application/json" \
  -d '{"results": [{"name": "Ali", "score": 90}, {"name": "Sara", "score": 75}], "query": "SELECT * FROM employees"}'
```

Challenges & Solutions

Challenge	Solution
User accidentally drops a table	Use <code>confirm: false</code> by default
Gemini generates invalid SQLite syntax	Try/catch in <code>execute_schema_sql()</code> returns the error
Insights not making sense	Send at least 5-10 rows for meaningful insights
ALTER TABLE doesn't work in SQLite	SQLite only supports <code>ADD COLUMN</code> for ALTER — note as limitation
Chart not generating	Make sure matplotlib is installed: <code>pip install matplotlib</code>

🌀 Integration: How All Modules Connect

TYPICAL WORKFLOW:

- 1. Habiba uploads students.csv
POST /file/upload → Table "students" created in SQLite

2. Aiza queries the data

POST /sql/query → {"question": "How many students got A grade?"}

→ AI makes SQL → runs on SQLite → returns results + chart
3. Aqsa gets insights on those results

POST /schema/insights → {"results": [...], "query": "..."}

→ AI analyzes → returns insights + chart
4. Rehan handles document questions separately

POST /rag/upload → uploads PDF

POST /rag/query → {"question": "...", "document": "..."}

→ RAG finds answer from PDF

✔ Final Checklist for Each Member

Before the deadline (Saturday 2nd May, 12:00 PM):

- ☐ Your module runs without errors
- ☐ All Flask routes are working
- ☐ You've tested with Postman or curl
- ☐ `__init__.py` is in your folder
- ☐ Code has comments explaining each function
- ☐ Your branch is pushed to GitHub
- ☐ You can explain your module in 2 minutes for viva
- ☐ `.env` is in `.gitignore` (never committed)

📖 Quick Reference: All API Endpoints

Method	Endpoint	Who	What
POST	<code>/file/upload</code>	Habiba	Upload CSV/Excel → creates DB table
GET	<code>/file/tables</code>	Habiba	List all DB tables
POST	<code>/sql/query</code>	Aiza	Ask question → get SQL + results + chart
GET	<code>/sql/schema</code>	Aiza	View current DB schema
POST	<code>/rag/upload</code>	Rehan	Upload PDF/PPT for RAG
POST	<code>/rag/query</code>	Rehan	Ask question from document

Method	Endpoint	Who	What
POST	/schema/modify	Aqsa	Create/Alter table via English
GET	/schema/show	Aqsa	View full DB schema details
POST	/schema/insights	Aqsa	Get AI insights + chart from results
POST	/schema/drop	Aqsa	Drop a table (with confirmation)